

Anexo 12 — Saludo procedural (sin RL)

Archivo: Scripts/rl/procedural/wave.py

Para que sirvio:

Coreografía del saludo escrita como curvas de control en el tiempo (4 fases: retraer codo, girar muñeca, abrir/cerrar dedos, volver). Es aditiva: escribe los actuadores del brazo elegido sin interferir con el resto del motor de animacion.

```
1 | """Saludo procedural del Pit Droid (sin RL).
2 |
3 | Coreografía especificada por el usuario:
4 |     1. Retrae el forearm (codo flexionado hacia el cuerpo).
5 |     2. Gira la muñeca medio giro (~90°).
6 |     3. Empieza a volver, abriendo y cerrando los dedos 2 veces.
7 |     4. Vuelve a pose de reposo.
8 |
9 | Duración total ≈ 4 s. Es ADITIVO: si el saludo está activo, escribe los joints
10 | del brazo elegido (izq o der); el resto de capas del Animation Engine siguen
11 | controlando sus joints sin conflicto.
12 |
13 | Uso:
14 |     wave = WaveAnimation(side="left", model=model)
15 |     wave.trigger(sim_time)
16 |     ...
17 |     ctrl_dict = wave.get_ctrl(sim_time) # {actuator_name: ctrl_value} o {}
18 | """
19 |
20 | from __future__ import annotations
21 | import math
22 |
23 |
24 | # Constantes de la coreografía. Los tiempos son MAS LENTOS que el v1 original
25 | # porque los actuadores del brazo (forearm forcerange ±2.11, wrist ±0.21)
26 | # tienen limitaciones fisicas reales – el wrist en particular es muy debil
27 | # y necesita tiempo para girar.
28 | DURATION_S = 6.0
29 |
30 | # Fase 1: retraer forearm (largo lento)
31 | T_RETRACT_START = 0.0
32 | T_RETRACT_END   = 1.5
33 |
34 | # Fase 2: girar muñeca (mas tiempo para que llegue al pico)
35 | T_WRIST_START   = 0.6
36 | T_WRIST_END     = 4.5
37 |
38 | # Fase 3: volver
39 | T_RETURN_START  = 1.8
40 | T_RETURN_END    = 5.5
41 |
42 | # Fase de dedos (2 ciclos durante la vuelta)
43 | T_FINGERS_START = 1.8
44 | T_FINGERS_END   = 5.3
45 | FINGER_CYCLES   = 2
46 |
47 | T_SETTLE_END    = DURATION_S
48 |
49 | # Amplitudes (porcentaje del rango del actuador).
50 | # v3: saludo MAS contenido (pedido del usuario): gira menos el codo, no levanta
51 | # tanto la mano, y casi no mueve el hombro hacia atras.
52 | FOREARM_RETRACT_AMOUNT = 0.45 # antes 0.95 – gira menos el codo, mano mas baja
53 | WRIST_HALF_TURN_AMOUNT = 1.00 # el giro de muñeca ES el gesto del saludo, se mantiene
54 | LEVER_OPEN_AMOUNT      = 1.00 # dedos abriendo/cerrando, se mantiene
55 | SHOULDER_LIFT_AMOUNT   = 0.10 # antes 0.50 – casi no tira el hombro para atras
56 |
57 |
58 | # Map de nombres de actuadores por lado
59 | ACTUATORS = {
60 |     "left": {
61 |         "shoulder": "act_LeftShoulderArm",
62 |         "forearm":  "act_LeftForearm",
63 |         "wrist":    "act_LeftWrist",
64 |         "lever":    "act_LeftLever_Slider",
65 |     },
66 |     "right": {
67 |         "shoulder": "act_RightShoulderArm",
68 |         "forearm":  "act_RightForearm",
69 |         "wrist":    "act_RightWrist",
70 |         "lever":    "act_RightLever_Slider",
71 |     },
72 | }
73 |
```

```

74 |
75 | def _smoothstep(x: float) -> float:
76 |     """Curva  $3x^2 - 2x^3$  para transiciones suaves entre 0 y 1."""
77 |     x = max(0.0, min(1.0, x))
78 |     return x * x * (3.0 - 2.0 * x)
79 |
80 |
81 | def _ramp(t: float, t_start: float, t_end: float) -> float:
82 |     """Devuelve 0 antes de t_start, 1 después de t_end, smoothstep en medio."""
83 |     if t_end <= t_start:
84 |         return 0.0
85 |     return _smoothstep((t - t_start) / (t_end - t_start))
86 |
87 |
88 | def _phase_in_window(t: float, t_start: float, t_end: float) -> float | None:
89 |     """Fase 0..1 si t está en la ventana; None fuera."""
90 |     if t < t_start or t > t_end:
91 |         return None
92 |     if t_end <= t_start:
93 |         return None
94 |     return (t - t_start) / (t_end - t_start)
95 |
96 |
97 | class WaveAnimation:
98 |     """Saludo procedural. Stateful para soportar 'trigger' y consultas por step.
99 |
100 |     Resuelve ctrl_lo y ctrl_hi de cada actuador al construirse (necesita el
101 |     'mujoco.MjModel'). En cada step devuelve un dict con los valores absolutos
102 |     a comandar para los 4 actuadores del brazo elegido.
103 |     """
104 |
105 |     def __init__(self, side: str, model):
106 |         if side not in ("left", "right"):
107 |             raise ValueError("side debe ser 'left' o 'right'")
108 |         self.side = side
109 |         self.act_names = ACTUATORS[side]
110 |
111 |         # Resolver ids y rangos de los actuadores
112 |         import mujoco
113 |         self._ids = {}
114 |         self._ranges = {}
115 |         for role, name in self.act_names.items():
116 |             aid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, name)
117 |             if aid < 0:
118 |                 raise ValueError(f"actuador {name!r} no existe en el modelo")
119 |             self._ids[role] = aid
120 |             lo, hi = float(model.actuator_ctrlrange[aid, 0]), float(model.actuator_ctrlrange[aid, 1])
121 |             self._ranges[role] = (lo, hi)
122 |
123 |         # Estado de la animación
124 |         self._start_time: float | None = None
125 |         self._active = False
126 |
127 |         # ----- API publica -----
128 |
129 |         def trigger(self, sim_time: float) -> None:
130 |             """Dispara la animación. Si ya estaba activa, reinicia desde 0."""
131 |             self._start_time = sim_time
132 |             self._active = True
133 |
134 |         @property
135 |         def active(self) -> bool:
136 |             return self._active
137 |
138 |         def get_ctrl(self, sim_time: float) -> dict[str, float]:
139 |             """Devuelve dict {actuador_name: ctrl_value} para los 4 joints del brazo
140 |             si la animación está activa. Si no, dict vacío.
141 |
142 |             El llamador (Animation Engine) integra estos valores en el vector de
143 |             control global, sobrescribiendo solo estos actuadores.
144 |             """
145 |             if not self._active or self._start_time is None:
146 |                 return {}
147 |             t = sim_time - self._start_time
148 |             if t > DURATION_S:
149 |                 self._active = False
150 |                 return {}
151 |
152 |             # --- forearm: retract con smoothstep, vuelta con smoothstep ---
153 |             # retract: 0 → FOREARM_RETRACT_AMOUNT entre T_RETRACT_START..T_RETRACT_END
154 |             # vuelta: FOREARM_RETRACT_AMOUNT → 0 entre T_RETURN_START..T_RETURN_END
155 |             retract_amount = _ramp(t, T_RETRACT_START, T_RETRACT_END) * FOREARM_RETRACT_AMOUNT
156 |             return_amount = _ramp(t, T_RETURN_START, T_RETURN_END) * FOREARM_RETRACT_AMOUNT
157 |             forearm_pct = retract_amount - return_amount # neto
158 |

```

```

159 | # --- wrist: half turn ida y vuelta (sin entre T_WRIST_START..T_WRIST_END) ---
160 | wrist_phase = _phase_in_window(t, T_WRIST_START, T_WRIST_END)
161 | if wrist_phase is None:
162 |     wrist_pct = 0.0
163 | else:
164 |     wrist_pct = WRIST_HALF_TURN_AMOUNT * math.sin(math.pi * wrist_phase)
165 |
166 | # --- lever: dos ciclos completos durante la vuelta ---
167 | fingers_phase = _phase_in_window(t, T_FINGERS_START, T_FINGERS_END)
168 | if fingers_phase is None:
169 |     lever_pct = 0.0
170 | else:
171 |     # 0 → 1 → 0 → 1 → 0 en N ciclos
172 |     lever_pct = LEVER_OPEN_AMOUNT * 0.5 * (
173 |         1.0 - math.cos(2.0 * FINGER_CYCLES * math.pi * fingers_phase)
174 |     )
175 |
176 | # --- shoulder: offset MINIMO para que el codo "asome" sin pegarse al cuerpo ---
177 | # v3: amplitud reducida (SHOULDER_LIFT_AMOUNT) para que casi no tire el hombro atras.
178 | shoulder_pct = SHOULDER_LIFT_AMOUNT * (
179 |     _ramp(t, T_RETRACT_START, T_RETRACT_END) - _ramp(t, T_RETURN_START, T_RETURN_END)
180 | )
181 |
182 | ctrl = {
183 |     self.act_names["shoulder"]: self._pct_to_ctrl("shoulder", shoulder_pct),
184 |     self.act_names["forearm"]: self._pct_to_ctrl("forearm", forearm_pct),
185 |     self.act_names["wrist"]: self._pct_to_ctrl("wrist", wrist_pct),
186 |     self.act_names["lever"]: self._pct_to_ctrl("lever", lever_pct),
187 | }
188 | return ctrl
189 |
190 | # ----- helpers -----
191 |
192 | def _pct_to_ctrl(self, role: str, pct: float) -> float:
193 |     """Mapea pct ∈ [-1, 1] al ctrlrange del actuador.
194 |
195 |     - pct = 0 → midpoint (o 0 si el ctrlrange incluye 0)
196 |     - pct = +1 → ctrl_hi
197 |     - pct = -1 → ctrl_lo
198 |     Para el lever (rango 0..max), pct positivo abre (pct=1 → ctrl_hi),
199 |     pct negativo se clipea a 0.
200 |     """
201 |     lo, hi = self._ranges[role]
202 |     if lo >= 0.0:
203 |         # rango [0, hi] (caso lever): solo pct positivo
204 |         return max(0.0, min(hi, pct * hi))
205 |     # rango bilateral [lo, hi]: pct=+1 → hi, pct=-1 → lo
206 |     if pct >= 0.0:
207 |         return min(hi, pct * hi)
208 |     return max(lo, pct * abs(lo))
209 |

```